

Group 1 Members & Contributions

- **Dwane Valencia**

- 1.2 Chosen SDG and Problem/Gaps
- 1.3 Current Technology
- 1.5 Research Objective
- 3.4 Evaluation Methods

- **Heart Sabino**

- Abstract
- 2.2 Event-Driven Architectures for Real-Time Systems
- 2.3 Comparative Analysis: Event-Driven vs. Request-Response Models
- 2.5 Synthesis and Research Gap
- 3.1 Research Design

- **Paul Joel Perez**

- 1.1 Background and Context
- 1.4 Proposed Study
- 1.6 Research Implications
- 3.2 Development Methods for Event-Driven Programming
- Proofread the draft (briefly)
- Double-checked References

- **William Datu**

- 2.1 The Evolution and Principles of Event-Driven Programming
- 2.4 Public Safety and Community Security and the Role of EDP
- 3.3 Data Gathering Procedure
- Double-checked References

All members contributed to the references as they wrote the paper.

SenyAlert: An Event-Driven System for Real-Time Emergency Hand Signal Detection and Timestamped CCTV Event Monitoring

Abstract (Heart)

The demand for automated security systems has created opportunities for integrating event-driven programming (EDP) and computer vision into monitoring. Traditional CCTV systems rely on human observation, making it difficult for operators to monitor camera feeds and identify emergency incidents. Existing emergency hand-signal detection systems focus on recognizing gestures but have limited integration with event logging, timestamp recording, camera identification, and alert generation. To address these limitations, this study proposes SenyAlert, an event-driven system designed for real-time emergency hand-signal detection and timestamped CCTV event monitoring. The system aims to convert a detected “Signal for Help” into a timestamped event by identifying the camera perspective, recording information, and triggering an alert. The study will follow a Developmental Research Design using a Design Science Research (DSR) approach. SenyAlert will implement a publish-subscribe architecture composed of a computer vision engine using OpenCV and MediaPipe, a WebSocket communication bridge, a Java Swing-based triage and dispatch application, and SQLite for event storage. Camera feeds will be processed to detect the sequence of an open palm, folded thumb, and closed fist associated with the “Signal for Help.” Once detected, the system will generate a JSON event containing information such as the camera ID, timestamp, and contextual data. Testing will progress from controlled webcam conditions to semi-controlled CCTV environments and, where applicable, a barangay CCTV environment. System performance will be evaluated through detection latency, throughput, resource utilization, scalability, reliability, load testing, stress testing, and fault-tolerance testing. The expected

outcome is an event-driven CCTV monitoring system that facilitates faster identification and review of emergency incidents while reducing reliance on manual observation. Ultimately, SenyAlert aims to contribute to safer inclusive communities while supporting Sustainable Development Goal 11: Sustainable Cities and Communities.

Keywords

SenyAlert, Event-Driven Programming, Event-Driven Architecture, Computer Vision, Emergency Hand Signal Detection, SOS Handsign, CCTV Monitoring, Real-Time Systems

1.1 Background and Context (Paul)

Evolution of Software Architectures

Software engineering architectures have progressively shifted from centralized structures toward distributed, reactive models to address growing demands for concurrency and low latency. Traditional monolithic architectures bundle user interfaces, business logic, and data access into a single codebase and execution runtime (Cabane & Farias, 2024). While straightforward to construct initially, monolithic systems suffer from tight coupling, where an operational failure or high computational load in one module can degrade or halt the entire application. The subsequent transition to microservice architectures divided systems into independently deployable services that communicate across network boundaries. However, microservices traditionally rely on synchronous request-response communication (such as HTTP/REST), where a client service must block and wait for a downstream server to complete processing (Rahmatulloh et al., 2022). This introduces latency bottlenecks, tight temporal coupling, and cascading failures under heavy network loads. To resolve these limitations, modern systems have increasingly adopted event-driven architecture (EDA) and event-driven

programming (EDP), which decouple interacting components and replace blocking calls with asynchronous, push-based communication (Bellemare, 2020; Hohpe & Woolf, 2003).

Fundamental Principles of Event-Driven Programming

Event-driven programming organizes application execution around the production, detection, and consumption of state changes. The paradigm relies on four fundamental elements:

- **Events:** An event represents a discrete record of a significant state change or incident within the system, such as a threshold breach, sensor trigger, or user action. The event payload typically includes context metadata, such as identifiers, timestamps, and classification parameters.
- **Event Producers:** Components, sensors, or background processes that detect internal or external state changes and construct event notifications. Producers operate independently of consumers and emit event payloads without requiring knowledge of how or when downstream modules will react.
- **Event Brokers:** Brokers or transmission channels act as the decoupled intermediary transport layer. They ingest events published by producers and route them asynchronously to interested parties via persistent streams, message queues, or persistent sockets, eliminating synchronous polling.
- **Event Consumers:** Consumers are decoupled services, listener routines, or subscriber user interfaces that register interest in specific events. Upon receiving an event from the broker, consumers parse the payload and execute subsequent business logic, data persistence, or interface triage off the main communication path.

The shift toward event-driven programming is essential for domain areas requiring real-time data ingestion and processing, including the Internet of Things (IoT), financial ticker systems, smart city infrastructure, and automated video surveillance.

- **Responsiveness:** Asynchronous event pipelines ensure that intensive operations—such as I/O reads, database transactions, or computer vision frame evaluations—execute off the primary presentation or communication threads. This maintains continuous interface responsiveness and prevents system freezes during sudden data surges.
- **Loose Coupling:** By isolating producers from consumers through an intermediary broker, individual modules can fail, restart, or scale independently without breaking dependent components or stalling the entire pipeline.
- **Scalability:** Systems avoid the computational waste of repetitive polling cycles by utilizing persistent push channels, enabling them to handle thousands of concurrent sensor feeds, transaction streams, or video inputs with minimal transport overhead and reduced operational latency.

1.2 Chosen SDG and Problem/Gaps (Dwane)

Chosen Sustainable Development Goal (SDG)

SDG 11: Sustainable Cities and Communities

Statement of the Problem

CCTV systems are widely used for security and monitoring. However, reviewing recorded footage can be difficult and time-consuming, especially when an emergency event needs to be located. Low-quality camera footage can also affect the detection of emergency hand signals. In addition, many CCTV systems lack an automatic system to detect a “Signal for Help” and immediately identify it as an emergency.

Identified Gaps

- **Technological Gap:** Existing CCTV and hand-signal detection systems have limited integration with event-driven technology that can automatically detect an emergency hand signal, create an event, and record its timestamp for monitoring and review.
- **Knowledge Gap:** There is limited research on integrating emergency hand-signal detection into an event-driven CCTV workflow that supports real-time event monitoring and easier identification of emergency events in recorded footage.

1.3 Current Technology (Dwane)

Overview of Existing Technologies

Recent studies in computer vision have made progress in detecting distress gestures, specifically the "Signal for Help." Buccellato et al. (2023) have conducted research and built systems that use deep learning models to recognize hand movements. These systems analyze video feeds using tools such as hand tracking and image classification to detect specific hand gestures used during emergencies.

Capabilities and Strengths

- **Accurate Detection:** These systems can reliably track hand points and recognize the "Signal for Help" finger-fold in controlled video feeds.
- **Automated Recognition:** They prove that software can detect distress gestures without human assistance.

Limitations of Current Approaches

- **Isolated Systems:** Most existing tools only focus on recognizing the gesture. They do not connect directly to a larger monitoring network or an automated notification system.

- **Lack of Timestamped Events:** Current systems do not automatically create lightweight event records with exact start times, camera locations, and frame markers.
- **Manual Video Searching:** Because detection is not linked to an alert system, operators still need to manually look through hours of recorded video to locate the exact moment of an incident.
- **Impact of Camera Quality:** Many models work well only with clear, close-up camera angles and struggle when using real-world CCTV cameras that have lower resolution or poor lighting.

Existing systems address only the computer-vision side of hand-signal detection. SenyAlert builds upon this by connecting detection directly to an event-driven framework. Instead of just displaying a box on a screen, SenyAlert treats a detected "Signal for Help" as an immediate trigger that creates a timestamped event, sends an alert, and indexes the exact moment in the CCTV footage for quick and easy review.

1.4 Proposed Study (Paul)

This study proposes SenyAlert, an event-driven program designed to automate real-time distress signal detection and eliminate manual monitoring of passive video feeds, and perform security triage. Instead of forcing security or law enforcement to review hours of CCTV footage to painstakingly locate an incident, SenyAlert uses event-driven programming (EDP) to record the exact timestamp of an emergency and instantly alert third parties, such as campus security or law enforcement, enabling quick and efficient intervention.

EDP is highly suitable for this security domain due to the following architectural advantages:

- **Asynchronous & Non-blocking:** By isolating heavy computer vision processing to the edge, the system pushes lightweight JSON alerts to the Java Swing dispatch dashboard without blocking the Event Dispatch Thread (EDT), ensuring the user interface remains highly responsive during critical emergencies.
- **Loose Coupling & Scalability:** The event producers (Python-based camera nodes) and consumers (Java triage applications) operate independently over a network bridge. This makes it easy to scale the system across multiple cameras or integrate new edge appliances without altering the core triage logic.
- **Real-time Event Processing:** The architecture captures and reacts to the universal "Signal for Help" the exact second it is executed. It autonomously evaluates threat contexts to fire multi-tier responses, such as a silent watch event or an audible alarm, intercepting threats before they escalate.
- **Decoupled Data Flow:** Treating CCTV feeds as continuous event streams simplifies the integration of new data sources and dispatch services, enabling lightweight alerts to be routed seamlessly to specific local Barangay VAWC desks or security terminals without notifying the aggressor.

By leveraging a decoupled WebSocket bridge to handle real-time gesture telemetry, the proposed system aims to transform standard CCTV cameras into automated tripwires, thereby accelerating emergency response times and creating safer, more inclusive public spaces.

1.5 Research Objectives (Dwane)

General Objective

To design and develop SenyAlert, an event-driven framework for real-time emergency hand-signal detection and timestamped CCTV event monitoring.

Specific Objectives:

- **Analyze** the limitations of traditional CCTV systems regarding manual footage backtracking and the lack of automated alerts.
- **Design** an event-driven architecture that automatically converts a detected "Signal for Help" on screen into a timestamped emergency event.
- **Develop** a functional prototype that detects the hand gesture from the output display and logs the exact camera POV and time.
- **Evaluate** the system's performance in terms of response latency, detection accuracy across different camera types, and reliability during downtime.

1.6 Research Implications (Paul)

Theoretical Contributions

The study introduces a novel architectural design pattern that integrates computer vision with event-driven architecture and programming. It provides the framework for taking gesture recognition from a passive detection script to a reactive, event-driven program. By using a local WebSocket bridge to push lightweight JSON payloads, the architecture demonstrates that heavy edge-based vision processing can be decoupled from a GUI, which, on Java Swing, prevents the Event Dispatch Thread (EDT) from being blocked when it needs to service important desktop dashboards. Furthermore, the research introduces a cascading triage engine that autonomously evaluates contextual metadata to execute multi-tier event responses, such as choosing between a silent watch and an audible alarm, without requiring manual operator intervention.

Practical Contributions

SenyAlert provides a functional, edge-computed appliance that converts standard, passive CCTV cameras into real-time, automated tripwires. This eliminates the vulnerability of relying on human security guards to monitor dozens of static video feeds manually, mitigating operator fatigue and missed distress signals. The system establishes a critical, touchless reporting mechanism for victims of intimidation or domestic abuse who cannot safely access physical panic buttons or phones in the presence of an aggressor. These findings offer an evidence-based solution that can be deployed by campus security offices or local Barangay VAWC desks to intercept threats before they escalate. This directly supports SDG 11: Sustainable Cities and Communities by providing a scalable framework to ensure safe and inclusive public spaces.

2.1 The Evolution and Principles of Event-Driven Programming (William)

Application integration has never been fully solved; it keeps changing form. Early systems that needed to talk to each other usually used one of a few simple methods: a batch file that one program writes and another reads later, a shared database that both systems use so neither has to send anything to the other, or a direct call where one system reaches into another and waits for an answer (Hohpe & Woolf, 2003). Each of these approaches comes with the same problems: networks are slower and less reliable than a local function call, the systems on each end usually were not built to understand each other, and once two systems are linked this way, a change on one side tends to break the other (Hohpe & Woolf, 2003). Messaging emerged to get around this; instead of calling another system directly and waiting, an application sends a message and moves on, and whoever needs it can read it later, on their own time (Hohpe & Woolf, 2003). This is really where event-driven programming (EDP) comes from. Not one single

idea, but a slow shift away from systems that call each other and wait, toward systems that simply announce what happened and let others respond in kind.

What makes a system "event-driven" instead of just "message-based" is more specific, though the two overlap a lot. An event-driven system is built around producers that send out events and consumers that respond to them. Once the system gets big enough, some kind of broker or channel in between, so a producer never needs to know who, if anyone, is listening (Hohpe & Woolf, 2003). Bellemare (2020) describes this as a change in what happens to a piece of data after it is sent: in older messaging setups, a message is normally used once and thrown away, but in an event-driven system, the event is kept as a lasting record that can be stored, reused and read by more than one consumer, without the producer having to coordinate with any of them. This is what gives event-driven systems their loose coupling — a consumer can be added, removed, or taken down without the producer ever knowing, because all the producer does is publish, not deliver.

There is a second, related idea in the literature that's less about connecting systems and more about making sense of what they send. Luckham's work (2002) on complex event processing (CEP). Luckham points out that most events moving through a system are never examined closely: they pass back and forth as unrelated bits of communication, and only once they are gathered and linked together do they become something useful rather than noise (Luckham, 2002). CEP is basically a set of methods for doing that linking, spotting when a sequence or pattern of small events actually means that something bigger is happening, whether that is a network break-in, a performance issue, or, in this case, a specific hand-gesture sequence in front of a camera.

These three ideas—producers and consumers who do not need to know about each other, connected through a channel; events that last beyond a single use; and finding patterns across a series of events—align closely with the setup used in this study. The Vision Ingestion Engine is the producer: it does not stream raw video the whole time; it watches for a specific pattern (palm, thumb-fold, fist) across a run of frames and only sends out a JSON event once that pattern is confirmed — exactly the kind of event-linking Luckham describes. The WebSocket Communication Bridge is the channel that allows the producer to remain completely unaware of who, if anyone, is on the other end. Moreover, the Triage & Dispatch Application is the consumer, waiting until an event shows up instead of constantly checking the vision engine, the same wait and react relationship between producer and consumer that Hohpe and Woolf described, just built with a modern web protocol instead of a traditional message queue.

2.2 Event-Driven Architectures for Real-Time Systems (Heart)

Event-Driven Architecture (EDA) is an architectural approach in which system components respond to events generated by internal or external sources. It is particularly applicable to real-time systems because information can be processed as events occur, allowing different components to respond without requiring the entire data stream to be collected first. In IoT environments, where devices continuously generate data, event-driven approaches can support timely processing and coordination between distributed components. Khriji et al. (2022) demonstrated this through a cloud-based event-driven architecture for real-time data processing in wireless sensor networks. Their architecture processed data from distributed sensor nodes. It incorporated MQTT and Apache Kafka to facilitate communication and event processing, demonstrating EDA's suitability for systems that require continuous data transmission and real-time analysis.

Rosa-Bilbao et al. (2023) further demonstrated the application of EDA in IoT through the CEPEDALoCo framework, which integrates complex event processing to analyze and correlate heterogeneous data streams generated by IoT devices. The framework enables the detection of significant conditions from continuously generated data, illustrating how event-driven systems can transform incoming data into actionable events that trigger subsequent processing. This capability is relevant to real-time monitoring applications, where detecting a specific condition must trigger an immediate system response.

The principles of real-time event processing are also applicable to video surveillance. Karbalaie et al. (2022) reviewed automated event detection in surveillance videos. They identified challenges, including varying event durations, simultaneous activities, camera viewpoint changes, occlusion, background complexity, and the unstructured nature of real-world surveillance footage. Their review highlights the importance of accurate and efficient object detection and activity classification for identifying significant events in surveillance videos. These requirements are particularly relevant to CCTV-based systems that continuously monitor environments for potentially critical incidents.

Recent studies have also demonstrated the feasibility of real-time hand gesture recognition using computer vision and deep learning. Sahoo et al. (2022) developed a real-time hand gesture recognition system using a fine-tuned convolutional neural network, demonstrating the application of deep learning to recognize hand gestures from visual input. Alaftekin et al. (2024) similarly applied a YOLO-based object detection approach to real-time sign language recognition, demonstrating the capability of object detection models to identify hand-based visual signs. Rodríguez-Moreno et al. (2025) further explored a lightweight keypoint-based approach using medoid similarity, emphasizing computational efficiency for real-time hand gesture recognition. Together, these studies demonstrate the continued development of

computer vision techniques capable of recognizing hand movements while maintaining the responsiveness required for real-time applications.

Collectively, these studies establish the relevance of event-driven architecture, real-time data processing, CCTV event detection, and hand gesture recognition to the development of real-time monitoring systems. EDA provides an architectural approach for responding to events generated from continuous data streams. At the same time, research in surveillance and gesture recognition demonstrates the feasibility of extracting meaningful events from visual input. These findings support SenyAlert, in which an emergency hand signal detected from a CCTV feed can be treated as an event that triggers subsequent processes such as timestamp recording, CCTV event monitoring, and alert generation. Thus, integrating event-driven processing with real-time computer vision provides a suitable foundation for linking emergency hand-signal detection to timely monitoring and response.

2.3 Comparative Analysis: Event-Driven vs. Request-Response Models (Heart)

Traditional CCTV systems used for barangay security and emergency response generally depend on continuous monitoring, human interpretation, and coordination among personnel. Marahay (2026), in a qualitative study of barangay CCTV operations in Mandaluyong City, found that CCTV systems can improve situational awareness, crime investigation, and emergency response coordination. However, technological limitations, lack of continuous monitoring, insufficient personnel training, budgetary constraints, and coordination and governance concerns were identified. The study further emphasized that CCTV systems are most effective when supported by active monitoring, trained personnel, reliable infrastructure, and coordinated response mechanisms.

These findings highlight an important limitation of conventional CCTV monitoring: although cameras continuously collect visual information, the detection and interpretation of an emergency may still depend on human operators. In situations where an operator must monitor multiple camera feeds simultaneously, an incident may not be recognized or documented immediately. This creates an opportunity for an automated, event-driven approach in which a predefined emergency signal detected in a CCTV feed can automatically trigger an event and record its occurrence.

Event-driven architecture (EDA) offers an alternative approach to system communication and processing. Instead of requiring a component to continuously request information from another component and wait for a response, an event-driven system allows an event to trigger subsequent processing asynchronously. Rahmatulloh et al. (2022) empirically examined the utilization of EDA as an alternative to API-driven communication in microservice-based systems. Their results showed that the EDA implementation achieved a 19.18% improvement in response time and a 34.40% reduction in error rate compared with the API-driven approach. However, the event-driven implementation also required greater CPU resources, demonstrating that improvements in responsiveness may involve computational trade-offs.

The performance benefits of EDA should therefore not be interpreted as universally superior to traditional architectures. Cabane and Farias (2024) conducted an empirical comparison between an event-driven implementation and a monolithic implementation, using metrics such as CPU usage, memory consumption, response time, throughput, and network traffic. Their findings indicated that, under their experimental conditions, the monolithic architecture consumed fewer computational resources and achieved better response times than the event-driven implementation. The authors consequently emphasized that architectural decisions

should be based on a system's specific requirements and characteristics rather than assuming that event-driven architecture will always deliver better performance.

For SenyAlert, the value of an event-driven approach is therefore not solely based on achieving the lowest response time or computational cost. Rather, its primary advantage is the ability to automatically respond to a detected event without requiring continuous human initiation. When the system recognizes a predefined emergency hand signal in a CCTV feed, the detection can trigger subsequent processes, such as recording the detection, assigning a timestamp, and generating an alert for appropriate personnel. This creates a more proactive monitoring mechanism than a conventional workflow, in which CCTV footage must first be observed and interpreted by a human operator.

Therefore, the proposed SenyAlert system can be viewed as an automated event-detection layer that complements existing CCTV operations rather than completely replacing human monitoring. Marahay (2026) highlights the importance of active monitoring and trained personnel in barangay CCTV operations, while studies on event-driven architecture demonstrate the potential of asynchronous event processing to improve responsiveness and system decoupling, alongside associated computational and implementation trade-offs. SenyAlert addresses this gap by transforming a recognized emergency hand signal into a timestamped system event, enabling potentially important incidents to be automatically identified and documented. In this way, the proposed architecture seeks to reduce dependence on continuous manual observation while retaining human personnel as part of the overall emergency-response process.

2.4 Public Safety and Community Security and the Role of EDP (William)

The specific trigger behavior for this system is the internationally known "Signal for Help" gesture (palm shown, thumb folded into the palm, then fingers closed over the thumb), made so a person can quietly ask for help on video without tipping off someone nearby who might be threatening them. Because the gesture looks harmless to onlookers, it can go unnoticed by anyone who might otherwise try to stop the person from asking for help, though that also means someone who already knows the sign could recognize it too.

Objective 1 (Analysis) — Challenges in the domain. Buccellato et al. (2025) in a study titled *"From Detection to Intervention: An End-to-End System for Recognizing the 'Signal for Help' Gesture in Real-Time,"* analyzed the challenge of detecting the gesture reliably in real-world conditions — low-quality footage, crowded scenes, and the need to run in real time without heavy computational resources.

Objective 2 (Design) — Architectures proposed for the domain. Buccellato et al. (2025) proposed an on-device architecture using NVIDIA Jetson Orin Nano and Jetson AGX Orin boards with native MediaPipe support to cut latency and avoid sending video to a central server. Del Castillo (2025), *"A Robot-Based Surveillance System for Recognizing Distress Hand Signal,"* which proposes splitting the system into a separate detection module and an alerting layer embedded in a social robot.

Objective 3 (Development) — Implementation approaches. Viejo-López (2023), *"Hand SOS Gesture Detection by Computer Vision,"* which details an OpenCV + MediaPipe implementation with automatic alerting; IRJAEH (2025), *"SOS Detection: Distress Signal Recognition,"* which

adds a snapshot-and-timestamp step sent to a chosen contact; and Buccellato, Vacca, Azimi, De Sio, & Sterpone (2025), which implements a companion mobile app that notifies security personnel with geolocation data and a short video clip.

Objective 4 (Evaluation) — Benchmarks reported in similar systems. Riego del Castillo, Sánchez-González, González-Santamarta, & Rodríguez-Lera (2025), reporting 96.43% accuracy for their social-robot implementation.

2.5 Synthesis and Research Gap (Heart)

Existing studies have established the feasibility of using computer vision and artificial intelligence for real-time hand gesture recognition and emergency gesture detection. Buccellato et al. (2024), for example, developed a real-time “Signal for Help” recognition system designed to detect emergency gestures from surveillance footage. Their system combines person tracking, hand feature extraction, and machine-learning classification to identify gestures, demonstrating the potential of integrating computer vision with surveillance systems for emergency detection.

However, a knowledge gap remains in the integration of emergency gesture recognition with a complete automated CCTV monitoring and response workflow. Existing research demonstrates that a system can recognize a “Signal for Help.” However, there is limited research addressing what happens immediately after the gesture is detected, particularly in terms of automatically creating a timestamped event record, identifying the corresponding camera perspective, indexing the incident for later review, and triggering an automated alert. Thus, the existing body of research focuses more heavily on the recognition and classification of gestures than on

treating recognized gestures as actionable events within an event-driven monitoring architecture.

A geographical gap is also evident in the Philippine context. Philippine researchers have previously explored computer-vision-based hand gesture recognition. Ilao et al. (2007), for instance, developed the Vision System for Hand Gesture Recognition (VISOR) at De La Salle University, which used computer vision to detect and track predefined hand gestures. However, this study was published in 2007, indicating that foundational Philippine research on computer-vision-based hand gesture recognition dates back nearly two decades. More recent work by Jarabese et al. (2021) demonstrated the continued application of hand gesture recognition in the Philippines through a real-time Filipino Sign Language recognition system based on a convolutional neural network. The study used a dataset of Filipino Sign Language gestures and achieved up to 95% accuracy, demonstrating the feasibility of real-time gesture recognition in a Philippine setting.

Although these Philippine studies demonstrate that hand gesture recognition has been explored locally, there remains limited research specifically applying the technology to emergency “Signal for Help” detection through CCTV. Furthermore, the identified Philippine studies primarily focus on gesture recognition itself rather than integrating recognition with automated emergency-event logging, CCTV monitoring, timestamp recording, and alert generation. This indicates a need for more recent Philippine research that extends existing gesture-recognition technologies toward practical emergency monitoring and response applications.

Therefore, the study addresses both a knowledge gap and a geographical gap. From a knowledge perspective, it seeks to extend existing research from simple gesture recognition to

an event-driven system in which a detected emergency gesture automatically becomes a timestamped, actionable CCTV event. From a geographical perspective, it contributes to the relatively limited body of Philippine research applying computer vision and hand gesture recognition specifically to emergency surveillance and response. SenyAlert addresses these gaps by integrating real-time “Signal for Help” detection with an event-driven CCTV monitoring framework that records the time and camera perspective of a detected incident and initiates an automated alert.

3.1 Research Design (Heart)

Approach: Design Science Research (DSR) approach

This study will employ a Developmental Research Design, specifically a design-and-development approach, to systematically design, develop, and evaluate SenyAlert, an event-driven system for real-time emergency hand-signal detection and timestamped CCTV event monitoring. Richey and Klein (2005) describe developmental research as an approach concerned with the systematic study of development processes and the creation of knowledge through development practice. The authors emphasize that developmental research involves defining a problem, establishing research procedures, collecting and analyzing data, and evaluating the resulting development.

The study also adopts principles of Design Science Research (DSR) because its primary output is a functional technological artifact. Hevner et al. (2004) explain that design science research focuses on creating innovative artifacts to address identified problems and gaining knowledge through the building, application, and evaluation of those artifacts. This applies to SenyAlert because the study not only investigates the limitations of existing CCTV and emergency gesture

detection systems but also develops and evaluates a working event-driven system to address those limitations.

The research will therefore follow a structured developmental process consisting of the following phases:

1. Problem Identification and Motivation

The researchers will identify the limitations of existing CCTV monitoring and emergency hand-signal detection systems, particularly the difficulty of manually locating emergency events in recorded footage and the limited integration between gesture detection and automated event monitoring. These problems and their corresponding research gaps are discussed in Chapter 1.

2. Definition of Objectives for a Solution

Based on the identified problems and research gaps, the researchers will establish the objectives that the proposed system must accomplish. These include designing an event-driven architecture, developing a functional emergency hand-signal detection system, generating timestamped events, and evaluating the system's performance.

3. Design and Development

The researchers will design and develop the SenyAlert system using an event-driven architecture. The system will integrate the computer vision detection component, a WebSocket communication bridge, a Java-based triage and dispatch application, and SQLite event storage. The detailed architecture and development procedures are presented in Section 3.2.

4. Demonstration

The developed system will be demonstrated through staged testing scenarios using different camera sources and environmental conditions. Testing will progress from a controlled webcam environment to a semi-controlled CCTV environment and, where

applicable, a live community CCTV environment. This progression will demonstrate the system's ability to detect emergency hand signals and generate corresponding timestamped events under increasingly realistic conditions.

5. Evaluation

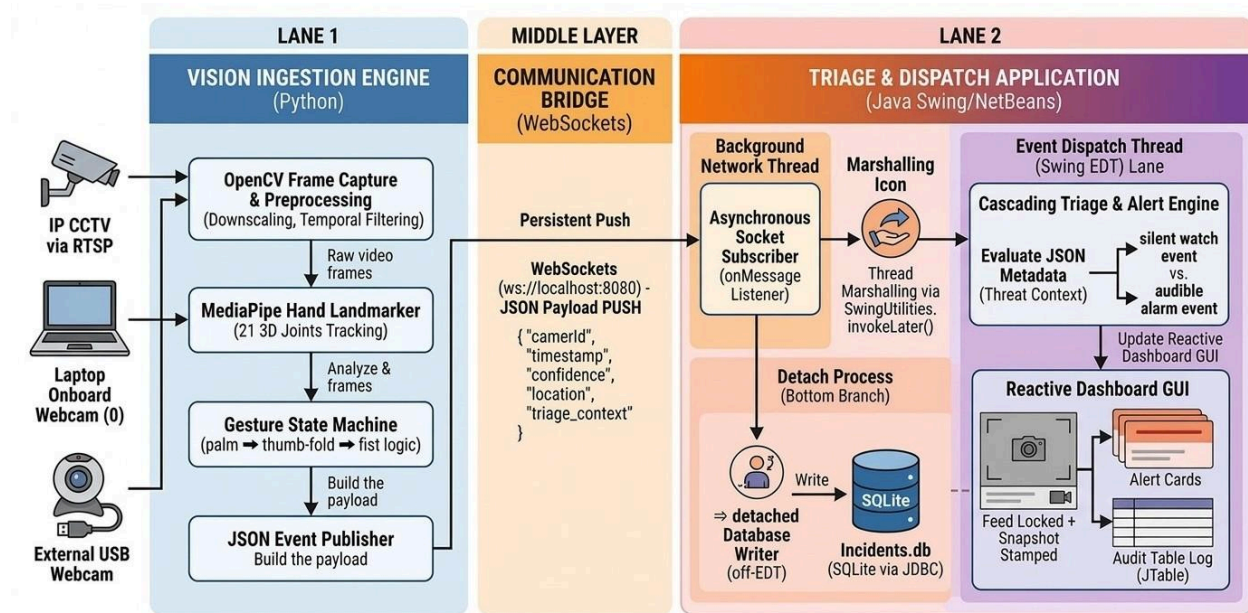
The developed artifact will be evaluated using defined performance criteria, including detection latency, throughput, resource utilization, scalability, and reliability. Load testing, stress testing, and fault-tolerance testing will be conducted to determine whether the system can process emergency events effectively and preserve event information during system failures. The detailed evaluation procedures are presented in Section 3.4.

6. Communication

The researchers will document and communicate the system's development process, implementation, evaluation results, findings, limitations, and implications. The results will be presented as evidence of the system's effectiveness in addressing the identified problem and as a basis for future improvements and related research.

3.2 Development Methods for Event-Driven Programming (Paul)

System Architecture



SenyAlert implements a decoupled, event-driven Publisher/Subscriber model that separates the edge computer vision processing from the Java graphical alert handling. The architecture consists of the following components, split into 4 layers:

Components:

- **Event Producers:** IP and RTSP CCTVs are the main event producers that record an area and send video frames to the edge engine. Other video-recording devices, such as external webcams and built-in laptop webcams, can also send to the edge engine. Any IoT device that records video frames can send raw data to SenyAlert with IP or RTSP.
- **Edge Engine:** Video frames from IP or RTSP CCTVs are captured by OpenCV on an edge engine coded in Python. Video frames from other devices, such as webcams, go straight to the edge engine. These frames are then processed by the edge engine using Google MediaPipe's Hand Landmarker to track 21 3D hand joint coordinates, which are included in the event payload.

- **Event Broker (Communication Bridge):** The Python edge engine pushes lightweight JSON event payloads over an asynchronous WebSocket connection (for example, `ws://localhost:8080`). Some of the values in the payload include "cameraId", "timestamp", "confidence", "location", and "triage_context". This enforces event-driven architecture through immediate response, minimal HTTP polling, and low local transport latency.
- **Event Consumers (Triage & Dispatch Application):** A desktop app houses the dashboard, powered by Java Swing. A background network thread continuously listens for incoming JSON payloads. To prevent GUI freezing, it uses `SwingUtilities.invokeLater()` to safely marshal parsed event payloads to the Event Dispatch Thread (EDT). Once these events are processed, the GUI dashboard on the desktop app updates, executing triage and triggering alerts.
- **Event Store (Data Persistence):** A detached database writer sends data to an embedded SQLite database (`incidents.db`) and stores data via JDBC. Database operations, such as logging timestamps and incident history, are executed asynchronously on a detached background worker thread to keep the EDT free.

Programming Paradigm & Implementation

- **Language/Framework:** The vision engine is built with Python, OpenCV, and MediaPipe. The desktop application that hosts the dashboard and security system is developed in Java (JDK 17+) using the Swing framework.
- **Event Processing:** SenyAlert relies on an asynchronous publish-subscribe workflow. Once the hand gesture sequence is confirmed, a JSON payload is pushed to the Java application. The Java application uses a cascading triage engine on the EDT to evaluate threat-context metadata, such as location, number of people present in the room, and proximity to the hand gesture. This context determines whether to fire a discrete

`SilentAlertEvent` or an `AudibleAlarmEvent`, as well as manual configuration by the specific user.

- **Workflow:** The workflow begins when OpenCV captures the raw video feed from any camera and applies downscaling and temporal filtering. MediaPipe then extracts hand landmarks and evaluates them using a state machine with three states: an open palm, a folded thumb, and a closed fist. Once evaluated, the Python engine constructs a JSON payload that includes the camera ID, timestamp, and context, then publishes it via WebSockets. The Java WebSocket listener receives the payload asynchronously and triggers an off-EDT database write to SQLite. Finally, the payload is dispatched to the EDT, which updates the reactive JTable audit log, locks the camera feed tile, and executes audio chimes if required by the triage level.

Development Tools

- **IDE:** VS Code is used for both Python and Java Swing GUI development.
- **Version Control:** Git and GitHub are used for repository management, with no CI/CD pipeline required.
- **Database Management:** DB Browser for SQLite is used to verify local schema changes and query updates in real time. Without it, the team cannot open any of the SQLite files.

3.3 Data Gathering Procedure (William)

Data source.

The system will not use an existing public dataset. Instead, data will come straight from live camera feeds run through the system itself (OpenCV frame capture → MediaPipe landmark extraction → gesture state machine output), so the data reflects real conditions instead of a cleaned-up test set.

Data Collection

Data will be gathered in three stages, each one less controlled than the last:

1. **Phase 1 — Webcam (controlled/development environment).** A regular laptop or desktop webcam will capture the first set of gesture sequences (palm → thumb-fold → fist) and non-gesture "background" activity. This stage happens under good conditions, decent lighting, one subject, close range, to check that the MediaPipe Hand Landmarker and gesture state machine catch the sequence correctly, and to set a baseline for accuracy and speed before adding real-world noise.
2. **Phase 2 — Computer shop CCTV (semi-controlled, real-world hardware).** The system will then run against an actual IP CCTV/RTSP feed at a computer shop. This brings in real variables missing from Phase 1: lower resolution and frame rate, a wider camera angle, changing light, more distance from subject to camera, and multiple people and background movement in frame. This phase checks whether the OpenCV preprocessing (downscaling, temporal filtering) and the gesture state machine still hold up once the camera source matches the actual hardware the system is built for, and it is the first time the whole pipeline (WebSocket push → triage dashboard → SQLite logging) runs start to finish on real, non-staged footage.
3. **Phase 3 — Barangay deployment (uncontrolled, live community environment).** The last stage tests the system on live CCTV already installed in a barangay (local community), matching the setting it is actually meant for. This is the least controlled stage — several people at once, unpredictable foot traffic, outdoor or mixed lighting, and cameras placed wherever they already are — and it is where the Cascading Triage & Alert Engine's ability to tell a real silent-watch/audible-alarm event apart from everyday activity gets tested hardest, along with how usable the Reactive Dashboard GUI is for whoever is watching it.

3.4 Evaluation Methods (Dwane)

Evaluation Criteria & Metrics

Performance Metrics

- **Latency:** Measures the time taken from Start Time (the exact moment the display frame showing the hand signal is captured from the output device) to event-processing completion (when the system detects the gesture, tags the specific camera POV, and logs the exact timestamp).
- **Throughput:** Measures the number of display frames analyzed and alert events processed per second without causing display lag or missing brief hand signals.
- **Resource Utilization:** Measures the CPU and RAM usage of the system during active detection. Different camera hardware types—a built-in laptop webcam, an external webcam, and a CCTV camera—will be tested to compare their impact on detection accuracy and system resource consumption.

Scalability

The system is scalable. It will be evaluated using an RTSP system that displays a 4-camera split-screen layout on a single output monitor to confirm that SenyAlert can consistently monitor all 4 camera views simultaneously and correctly identify which specific camera POV triggered the emergency event.

Reliability

Evaluates how the system handles service failures or downtime (such as the alert logger or monitoring dashboard crashing). It verifies that the exact timestamp of the detected emergency signal is safely captured and stored locally or buffered in the event queue, so that even if the system goes down, the timestamp is never lost and is immediately recovered upon restart.

Evaluation Procedure

- **Scenario 1: Load Testing (Standard Operating Load)**
 - a. Run the screen-detection pipeline on a normal CCTV feed displaying occasional hand signals. Measure the baseline latency from initial frame capture to timestamp save, while tracking average throughput (FPS).
- **Scenario 2: Stress Testing (Peak Load)**
 - a. Focus on a single camera POV and simulate peak stress by rapidly repeating emergency hand signals in quick succession, along with continuous hand movements. This tests the system's maximum processing speed, checking if event queues become bottlenecked, if frames drop, or if duplicate/delayed timestamps are generated on that single feed.
- **Scenario 3: Fault Tolerance Test**
 - a. Simulate a sudden system crash or service failure right as an emergency hand signal is recognized on screen. Check the system logs after rebooting to verify that the timestamp was successfully saved and preserved during the crash without data loss.

Comparative Analysis

- **Baseline System (Traditional Request-Response Model):** A polling-based setup where the system checks the screen at fixed intervals (e.g., every 1–2 seconds) to see if a hand gesture was detected.
- **Proposed System (Event-Driven Model):** SenyAlert's event-driven architecture, where detecting a hand sign immediately generates and pushes a timestamped event with the camera POV identifier.

- **Comparison:** Both systems will be tested on the CCTV RTSP display to compare alert latency, CPU usage, and timestamp accuracy.

References

- Alaftekin, M., Pacal, I., & Cicek, K. (2024). Real-time sign language recognition based on YOLO algorithm. *Neural Computing and Applications*, 36, 7609–7624. <https://doi.org/10.1007/s00521-024-09503-6>
- Azimi, S., De Sio, C., Carlucci, F., & Sterpone, L. (2023). Fighting for a future free from violence: A framework for real-time detection of "Signal for help". *Intelligent Systems with Applications*, 17, Article 200174. <https://doi.org/10.1016/j.iswa.2022.200174>
- Buccellato, F., Sio, C.D., Vacca, E., & Azimi, S. (2024). Enhancing Security of Smart Cities with "Signal for Help" Recognition System. *2024 IEEE International Smart Cities Conference (ISC2)*, 1-6. <https://ieeexplore.ieee.org/document/11004211>
- Cabane, H., & Farias, K. (2024). On the impact of event-driven architecture on performance: An exploratory study. *Future Generation Computer Systems*, 153, 52–69. <https://doi.org/10.1016/j.future.2023.10.021>
- Del Castillo, V. R., Sánchez-González, L., González-Santamarta, M. Á., & Lera, F. J. R. (2024). A robot-based surveillance system for recognising distress hand signal. *Logic Journal of IGPL*, 33(5). <https://doi.org/10.1093/jigpal/jzae067>
- Hevner, Alan R., et al. "Design science in information systems research1." *MIS quarterly* 28.1 (2004): 75-106.
- Hohpe, G. and Woolf, B. (2003) Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley.
- Ilao, J. P., Enero, J. M., Gonzalo, K. D., Salvador, R. P., & Ticzon, E. I. (2007). Vision system for hand gesture recognition. *Third Humanoid, Nanotechnology, Information Technology, Communication and Control Environment and Management (HNICEM) International Conference* Retrieved from https://animorepository.dlsu.edu.ph/faculty_research/6925

- Jarabese, M. B. D., Marzan, C. S., Boado, J. Q., Lopez, R. R. M. F., Ofiana, L. G. B., & Pilarca, K. J. P. (2021). Sign to speech convolutional neural network-based Filipino sign language hand gesture recognition system. 2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC). 147–153. DOI 10.1109/ISCSIC54682.2021.00036
- Karbalaie, A., Abtahi, F., & Sjöström, M. (2022). Event detection in surveillance videos: A review. *Multimedia Tools and Applications*, 81, 35463–35501. <https://doi.org/10.1007/s11042-021-11864-2>
- Khriji, S., Benbelgacem, Y., Chéour, R., et al. (2022). Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks. *The Journal of Supercomputing*, 78, 3374–3401. <https://doi.org/10.1007/s11227-021-03955-6>
- Luckham, D. (2008). The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems. Lecture Notes in Computer Science. https://doi.org/10.1007/978-3-540-88808-6_2
- Mukesh, N., Nithis Bangaru, B., Vignesh Kumaran, A., & Athilakshmi, S. (2025). SOS detection: distress signal Recognition. *International Research Journal on Advanced Engineering Hub (IRJAEH)*, 3(03), 782–785. <https://doi.org/10.47392/irjaeh.2025.0109>
- Richey, R. C., & Klein, J. D. (2005). Developmental research methods: Creating knowledge from instructional design and development practice. *Journal of Computing in higher Education*, 16(2), 23-38.
- Rosa-Bilbao, J., Boubeta-Puig, J., & Rutle, A. (2023). CEPEDALoCo: An event-driven architecture for integrating complex event processing and blockchain through low-code. *Internet of Things*, 22, 100802. <https://doi.org/10.1016/j.iot.2023.100802>
- Rodríguez-Moreno, I., Freire-Obregón, D., Martínez-Otzeta, J. M., & Castrillón-Santana, M. (2025). Real-time hand gesture recognition: Lightweight keypoint-based approach with

medoid similarity. *Journal of Advances in Information Technology*, 16(4), 447–457.

<https://doi.org/10.12720/jait.16.4.447-457>

Sahoo, J. P., Jaya Prakash, A., Pławiak, P., & Samantray, S. (2022). Real-time hand gesture recognition using fine-tuned convolutional neural network. *Sensors*, 22(3), 706.

<https://doi.org/10.3390/s22030706>

The Power of Events: An introduction to complex event processing in distributed enterprise systems 0201727897 - DOKUMEN.PUB. (n.d.). dokumen.pub.

<https://dokumen.pub/the-power-of-events-an-introduction-to-complex-event-processing-in-distributed-enterprise-systems-0201727897.html>

Viejo-López, R., Del Castillo, V. R., & Sánchez-González, L. (2022). Hand SOS gesture detection by Computer vision. *Lecture Notes in Networks and Systems*, 22–29.

https://doi.org/10.1007/978-3-031-18409-3_3